

Programming Languages for AI Readiness and Creativity Enhancement in Education: A Systematic Literature Review

Joko Saefan

jokosaefan@gmail.com

Universitas PGRI Semarang <https://orcid.org/0000-0002-2810-9628>

Siti Wahyuni

Universitas Negeri Semarang <https://orcid.org/0000-0001-7237-8115>

Bambang Subali

Universitas Negeri Semarang <https://orcid.org/0000-0002-7427-4245>

Hartono

Universitas Negeri Semarang <https://orcid.org/0000-0001-5684-9212>

Systematic Review

Keywords: AI readiness, creativity, programming education, programming languages, learning approaches

Posted Date: August 25th, 2025

DOI: <https://doi.org/10.21203/rs.3.rs-7428847/v1>

License:   This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: The authors declare no competing interests.

Programming Languages for AI Readiness and Creativity Enhancement in Education: A Systematic Literature Review

Joko Saefan^{1*}, Siti Wahyuni², Bambang Subali², Hartono²

¹Phycis Education Department, Universitas PGRI Semarang, Jl. Sidodadi Timur No. 24, Semarang, 50125, Central Java, Indonesia.

²Physics Department, Universitas Negeri Semarang, Jl. Sekran Raya, Semarang, 50228, Central Java, Indonesia.

*Corresponding author(s). E-mail(s): jokosaefan@gmail.com;
Contributing authors: wahyuni.smg@mail.unnes.ac.id;
bambangfisika@mail.unnes.ac.id; hartono@mail.unnes.ac.id;

Abstract

Artificial intelligence (AI) readiness and creativity enhancement are central goals of 21st-century education, yet the research landscape regarding effective programming languages and learning approaches to achieve them remains fragmented. This systematic literature review, maps the empirical evidence from 24 peer-reviewed studies (2022-2025) selected from the Scopus and ERIC databases. The analysis reveals that the effectiveness of a programming language is not inherent but is critically determined by pedagogical design. These are particularly long-term intervention duration and an authentic project-based orientation. A complex relationship between the two outcomes was found, block-based environments in contextual projects emerge as an ideal nexus for fostering AI readiness while formally measuring creativity at the foundational level. Conversely, at the higher education level, a focus on deep AI readiness tends to be accompanied by more holistic and less measured reporting of creativity. The primary contribution of this review is a two-dimensional classification framework for programming languages and pedagogical approaches, which serves as a strategic map for educators and curriculum designers. This framework provides guidance for aligning tools and methods with specific learning goals. whether optimizing for measurable creativity or deep technical readiness, the guidance is more effectively prepare students for the AI era.

Keywords: AI readiness, creativity, programming education, programming languages, learning approaches

1 Introduction

Artificial intelligence (AI) has transformed the educational landscape, establishing programming as a core foundation of AI readiness for creating innovative future solutions (Wing 2006; Manorat et al. 2025). Programming requires students to write code with strict rules in syntax and resolve errors, which forms the basis for building stable and reliable AI systems (Adetiba et al. 2021). Creativity in programming education is cultivated through learning approaches such as Project-Based Learning (PBL), which encourages students to find innovative solutions for coding problems through authentic projects that resemble real-world challenges in the AI domain (Hmelo-Silver 2004). This approach allows students to develop both technical skills and creativity, which are essential for AI readiness by generating new ideas and original solutions through in-class exploration and collaboration (Runco and Jaeger 2012). Thus, the right learning approach bridges programming languages with creativity, preparing students to face the challenges of future AI development. However, the literature on how programming languages and learning approaches support AI readiness and creativity remains fragmented, necessitating a systematic review to synthesize the evidence and provide clear curricular guidance (Manorat et al. 2025).

The literature on programming education is still fragmented, with studies tending to separately address programming languages for technical purposes and learning approaches for creativity, without a clear integration to support AI readiness (Long and Magerko 2020; Su and Yang 2022). The majority of studies focus on the technical applications of AI, such as machine learning development, rather than on the role of programming languages as a foundation for AI readiness in K-12 education, thereby giving less emphasis to preparing students for future AI development (Lee et al. 2021; Su and Yang 2022). Learning approaches like Project-Based Learning (PBL) show potential for enhancing creativity through authentic projects involving code-based problem-solving, but their connection with programming languages to support AI readiness has not been deeply investigated (Long and Magerko 2020). Furthermore, the literature has insufficiently explored how these learning approaches can be integrated into formal and non-formal education to simultaneously strengthen creativity and AI readiness (Lee et al. 2021). These gaps affirm the need for a systematic review to map and unify the evidence on how programming languages and learning approaches can be integrated to support AI readiness and creativity, thereby providing coherent curricular guidance for education in the digital era.

This study aims to synthesize evidence on the programming languages and learning approaches used to support AI readiness while enhancing creativity in educational contexts. The scope includes peer-reviewed studies that feature programming activities and are not merely the use of ready-made tools. This was conducted in both formal and non-formal education and reported outcomes related to creativity or proxies for

AI readiness (Wing 2006; Shute et al. 2017). The study focuses on articles published from 2022 to August 2025, aligning with the "new normal" era characterized by the recovery of educational practices post-pandemic (Bond et al. 2021). The year 2022 was chosen as the starting point to capture the latest literature on AI readiness and creativity enhancement through programming in the context of post-COVID-19 educational transformation (Haleem et al. 2022). As a limitation, this review does not aim to benchmark the technical performance between languages, and findings regarding accessibility issues are presented descriptively in relation to curricular implementation. The search, selection, and reporting procedures follow the PRISMA guidelines to enhance the transparency and replicability of this systematic review (Page et al. 2021).

The analytical framework for this study is designed to be concise yet operational, structured through a three-tier classification across two dimensions. In the programming language dimension, we distinguish among core language, contextual programming, and other/emerging. Core languages are dominant in AI development due to their library ecosystems and community support. Contextual languages are relevant to specific domains or needs. Within the contextual language, we provide an additional sub-section, namely contextual-block-based, such as Scratch. Other/emerging groups are beginning to appear, but have limited adoption. This classification is drawn from the literature on the evolution of AI languages and the mapping of the Python ecosystem in machine learning practice (Adetiba et al. 2021; Raschka et al. 2020). In the learning approach dimension, we group core models, contextual models, and other/emerging models. Core models are composed of Project-Based Learning (PBL), Problem-Based Learning (PBL), and Inquiry-Based Learning (IBL), which are founded on evidence of effectiveness for higher-order skills. Contextual models are exemplified by STEAM and game-based learning that link programming with media or interdisciplinary subjects. Other/emerging models are exemplified by creative coding, maker education, and hybrid AI-labs as spaces for innovation. This grouping follows findings from reviews that affirm the role of authentic task design and the level of guidance in learning achievement (Hmelo-Silver 2004; Kokotsaki et al. 2016; Lazonder and Harmsen 2016; Plass et al. 2015; Quigley et al. 2017). This two-dimensional framework serves as the lens to extract, map, and synthesize the study findings, allowing the programming language and learning approach to be read consistently across educational contexts for AI readiness and enhancing creativity.

On a conceptual level, this article proposes and operationalizes a two-dimensional framework for interpreting the literature in a classification of programming languages and a classification of learning approaches. The language framework is derived from evidence on the AI programming ecosystem, such as the dominance of Python in machine learning and data science and the emergence of high-performance languages in numerical computing. Thus are enabling a consistent mapping of language choices in education (Raschka et al. 2020; Bezanson et al. 2017; Adetiba et al. 2021). The AI readiness indicator is operationalized into a "strong" or "weak" category by considering the programming tasks performed during the learning process. Meanwhile, the approach framework gathers student-centered models that have demonstrated effectiveness relevant to creativity development (Hmelo-Silver 2004; Kokotsaki et al. 2016;

Lazonder and Harmsen 2016; Plass et al. 2015; Quigley et al. 2017). The classification of creativity is based on the dimensions of process, trait, and product (Hu and Adey 2002). Articles that do not include a classification of creativity dimensions are categorized by their domain-specific context of creativity (OECD 2024).

On a methodological level, this article presents a transparent and replicable Systematic Literature Review (SLR) protocol through a cross-database search strategy, inclusion-exclusion criteria emphasizing peer-reviewed publications, a two-stage screening procedure and quality assessment, and a data extraction codebook that captures variables for language, approach, context, and outcomes. Reporting follows the latest SLR guidelines to ensure the selection and synthesis process can be audited and replicated by other researchers (Page et al. 2021; Higgins et al. 2019). The final result is a descriptive-thematic synthesis, including a cross-tabulation between language programming and learning approach, which can serve as a foundation for curriculum design and further research agendas on AI readiness and creativity in education.

To focus the review and maintain analytical consistency, the following research questions (RQ) were formulated to guide this study:

- RQ1 What programming languages are used in educational contexts to support AI readiness?
- RQ2 What learning approaches in programming are applied that characterize creativity?
- RQ3 How is the link between AI readiness and enhancing creativity supported by programming languages and learning approaches?

2 Theoretical Framework

2.1 Programming Languages for AI Readiness

Artificial intelligence (AI) has become a primary driver of global technological innovation, demanding significant changes in education, particularly in preparing learners to face future challenges (Manorat et al. 2025). In this context, AI readiness encompasses the skills not only to utilize existing technologies but also to creatively and innovatively build and develop AI solutions (Adetiba et al. 2021). One of the core components of AI readiness is the mastery of programming languages relevant to AI development. The choice of language must consider factors such as technical relevance to modern AI trends, ease of learning through clear syntax, the availability of a supporting ecosystem of libraries and frameworks, and its accessibility in educational settings (Raschka et al. 2020; Türkmen et al. 2024). A review of the literature indicates that programming languages supporting AI readiness can be classified into three main groups: core AI readiness languages, contextual AI readiness languages, and emerging AI readiness languages. This classification is based on a comparative analysis of the technical characteristics, performance, and suitability of programming languages in various AI development contexts, both in industry and higher education (Adetiba et al. 2021; Bezanson et al. 2017).

2.1.1 Core AI Readiness Languages

Core AI readiness languages are considered the foundation of AI learning and development due to their mature library ecosystems, large communities, and widespread use in both industry and academia (Adetiba et al. 2021).

1. Python stands out due to its user-friendly syntax and vast AI/ML ecosystem, including numerical libraries like NumPy that facilitate efficient scientific computing (Harris et al. 2020). In the realm of deep learning, popular frameworks such as TensorFlow and PyTorch provide a robust ecosystem for education and research (Abadi et al. 2016; Paszke et al. 2019). A comprehensive review of the Python ecosystem in the AI field confirms this trend (Raschka et al. 2020).
2. C++ is relevant in scenarios requiring high performance, such as real-time system optimization and inference pipelines, as identified in surveys on neural network processing efficiency. The implementation of performance-based libraries like XGBoost demonstrates the importance of C++ for developing large-scale AI models (Chen and Guestrin 2016).
3. Julia emerges as a modern language for high-performance numerical computing, combining the productivity of a high-level language with speeds approaching C/C++ (Bezanson et al. 2017). Julia's ecosystem for AI is rapidly expanding through libraries like Flux.jl, which is specifically designed for deep learning (Innes 2018).

Overall, Python with an extensive ecosystem and high accessibility, C++ in performance and efficiency, and Julia through modern numerical computing, provide a powerful combination of core competencies to equip learners in designing, developing, and creatively optimizing AI solutions in higher education.

2.1.2 Contextual AI Readiness Languages

Languages in the contextual AI readiness category serve as a bridge between general programming education and AI applications. Although not "core languages" for cutting-edge AI development, Java, R, and MATLAB remain relevant due to their stability, library ecosystems, and broad application in education and industry (Adetiba et al. 2021). Block-based are taken into additional consideration in contextual language due to their popularity in educational applications.

1. Java is commonly used to teach OOP principles and software engineering; the objects-first approach demonstrates Java's role as a conceptual foundation before students transition to more dominant languages in AI. Furthermore, Java's ecosystem for data mining/ML shows a practical pathway to modeling that can be integrated into large-scale systems (Cooper et al. 2003; Hall et al. 2009).
2. R excels in statistical analysis and data exploration; packages like caret facilitate systematic modeling workflows for classification and regression, while the glmnet family extends its reach to modern regularization. This combination makes

- R effective for AI curricula that emphasize data understanding and statistical literacy before advanced model development (Kuhn 2008; Friedman et al. 2010).
3. MATLAB provides an integrated environment for numerical computing and matrix-based modeling. In engineering education, learning MATLAB has been shown to support the mastery of concepts and computational skills, making it relevant as a computational foundation that can be linked to AI topics (Higham and Higham 2017; Li and Huang 2017).
 4. Block-based programming languages, such as Scratch, Snap!, and Blockly, represent a significant pedagogical tool within the contextual language category. These visual programming environments are designed to lower the cognitive barrier for novice programmers by abstracting away complex syntax into interlocking graphical blocks (Resnick et al. 2009). This "low-threshold" approach allows learners, particularly in K-12 settings, to focus on computational concepts like sequencing, loops, and conditionals rather than struggling with typographical errors. While not used for professional AI development, their role in building AI readiness is foundational. They serve as a critical bridge to text-based programming by building students' confidence, fostering computational thinking skills, and allowing for creative expression in contextual projects like game design and storytelling (Weintrop and Wilensky 2015).

In summary, these four languages strengthen AI readiness through conceptual foundations, computational practices, and systems integration in helping students prepare for a smooth transition to more specialized AI ecosystems.

2.1.3 Other/Emerging AI Readiness Languages

Besides core and contextual languages, a number of other programming languages have the potential to support AI readiness, although their adoption in education is still limited. These languages often arise from corporate or community initiatives and are typically designed for very specific needs. Examples include Rust for safe and efficient computational applications, Scala for large-scale data processing, and Go for cloud-based services and distributed systems (Klabnik and Nichols 2018; Odersky et al. 2019; Donovan 2016). Although their scope of application in AI education is relatively small, the existence of these languages indicates future technological developments that could expand learning alternatives in the future.

2.2 Learning Approaches for AI Readiness and Creativity Enhancement

The selection of a learning approach is critical to the successful integration of programming to equip students with AI readiness while fostering creativity. Studies show that student-centered strategies can enhance computational competencies and creative outputs, especially when programming tasks are authentic and contextually relevant (Lye and Koh 2014; Ulger 2018; Wang et al. 2016). At the same time, the strengthening of computational thinking as a conceptual foundation supports creativity in programming activities (Shute et al. 2017; Wing 2006). Meta-analytic evidence also

confirms the effectiveness of IBL when designed with an adequate level of guidance, a crucial finding for designing programming instruction that targets both AI readiness and provides space for creation ([Lazonder and Harmsen 2016](#)).

2.2.1 Core Approaches

Core approaches include learning models that directly involve learners in the programming process through structured, product-oriented activities.

1. PjBL places learners on complex programming tasks with the goal of producing a real solution or product, thereby encouraging the integration of technical skills and creativity ([Kokotsaki et al. 2016](#)).
2. PBL focuses on solving ill-structured problems, which require learners to apply programming logic and computational thinking to formulate innovative solutions ([Hmelo-Silver 2004](#); [Ulger 2018](#)).
3. IBL provides space for learners to formulate questions, design computational experiments, and analyze results, which aligns with the principles of creative exploration in preparing for AI challenges ([Lazonder and Harmsen 2016](#)).

These core approaches are proven to strengthen higher-order thinking skills, increase confidence in using programming languages, and facilitate deep conceptual understanding for all crucial elements in preparing a generation of learners in the AI-based era.

2.2.2 Contextual Approaches

Contextual approaches situate programming education within a framework or medium that has direct relevance to real-world situations, allowing learners to connect technical skills with creative applications.

1. STEM/STEAM Education Models combine programming with the integration of science, technology, engineering, arts, and mathematics. These models enable students to work on cross-disciplinary projects that emphasize collaboration, innovation, and creativity, while strengthening their readiness for future technological challenges ([Yakman 2008](#); [Quigley et al. 2017](#)).
2. Game-Based Learning uses digital games or gamification elements to make programming education more engaging and interactive. This approach is proven to increase learning motivation, encourage active involvement, and strengthen creative problem-solving skills ([Plass et al. 2015](#)).
3. Domain-Specific Applications focus programming education on its application in specific fields, such as educational robotics, scientific simulations, or data analysis in healthcare. This specific context provides intrinsic motivation for students as they can see the direct benefits of the skills they are learning ([Benitti 2012](#); [Weintrop et al. 2016](#)).

2.2.3 Other/Emerging Approaches

Beyond core and contextual approaches, there are programming learning models that are new or experimental but show potential for strengthening AI readiness and creativity. Recent reviews and syntheses confirm these opportunities while emphasizing the need for careful pedagogical design to ensure their implementation is effective and sustainable in formal education contexts (Martin 2015; Rouse and Rouse 2022; Schad and Jones 2020). Some examples in this category include creative coding pedagogy, Maker education, and a hybrid AI-lab classroom. Creative coding pedagogy places programming as an expressive medium, allowing the practice of writing code to run parallel with aesthetic exploration and process reflection (Terroso and Pinto 2022; Blake-West et al. 2024). Maker education combines digital fabrication, micro-controllers, and design to link programming with tangible artifacts and authentic problem-solving. Hybrid AI-lab classroom integrates physical lab practices/artifacts with computational notebooks and cloud computing to support literate computing, replication, and data as model integration in AI-based programming tasks (Kluyver et al. 2016; Chen and Asta 2022; Lopes et al. 2022).

3 Methods

3.1 Design and Guidelines

This study is a systematic literature review reported in accordance with the PRISMA 2020 guidelines. From the outset, a protocol was established that defined the scope, research questions, inclusion/exclusion criteria, search strategy, and a two-stage screening procedure, namely title/abstract followed by full-text review. Data extraction was performed based on a codebook that mapped programming languages, learning approaches, creativity key, AI readiness proxies, and other supporting variables. The synthesis of results is presented descriptively. The methodological framework refers to the determination of scope and questions, inclusion criteria, and the search and selection of studies (Page et al. 2021; Thomas et al. 2019; McKenzie et al. 2019; Lefebvre et al. 2019).

3.2 Search Strategy

A parallel strategy was designed for the Scopus and ERIC databases with the core terms "creativ*" and "programming/coding" in an educational context. Terms for learning approaches were not included in the search phase to avoid bias; noise was handled during the title/abstract screening and full-text review. Filters were applied according to the time period and access policies described in the previous subsection. The results from both databases were combined and deduplicated before screening.

1. Scopus

- Initial query:
TITLE-ABS-KEY (creativ*) AND TITLE-ABS-KEY (programming OR coding) AND TITLE-ABS-KEY (education OR school* OR universit* OR classroom OR curriculum OR course*)

- Initial results: $n = 2,933$.
 - After applying filters (document type: article/proceedings, language: English, access: all open access, and year: 2022-2025), $n = 257$.
2. ERIC
- Initial query:
 (creative OR creativity) AND (programming OR coding) AND (education OR school OR schools OR university OR universities OR classroom OR classrooms OR curriculum OR curricula OR course OR courses)
 - Initial results: $n = 1,161$.
 - After applying filters (Publication Date 2022?2025, Publication Type "Journal Articles", Language English, and "Full text available on ERIC"), $n = 30$.

3.3 Inclusion and Exclusion Criteria

Data verification from ERIC and Scopus identified four duplicate articles ($n=4$). Consequently, 283 articles were retained for the screening process. In the next screening step, the titles and abstracts were examined based on the inclusion and exclusion criteria, yielding 99 articles eligible for retrieval. As 18 articles could not be downloaded, 81 articles remained for screening in the subsequent stage. The downloadable articles were further screened using the inclusion and exclusion criteria, as in the subsequent stage, resulting in 24 articles ready for synthesis.

1. Inclusion Criteria

- (a) Empirical studies from journal articles or conference proceedings in a formal or non-formal educational context.
- (b) Includes a programming activity actually performed by the learners (not just tool usage).
- (c) Reports on creativity characteristics and/or proxies for AI readiness, such as computational thinking, problem-solving, or the creativity of a product/artifact (Grover and Pea, 2013).
- (d) Published between January 2022 and August 2025, in English, with the full text available for verification (Scopus: open access; ERIC: full text available on ERIC).
- (e) Small-scale/pilot implementations are accepted as long as criteria (2)-(3) are met.

2. Exclusion Criteria

- (a) Framework/conceptual papers without classroom implementation.
- (b) Perception/adoption surveys without a programming task showing student activity.
- (c) Use of a platform/tool, e.g., AR/VR or a monitoring system, without learners writing code and without reported learning outcomes.

- (d) Non-computational uses of the term "coding/programming": qualitative coding/thematic analysis, mathematical/linear programming, media/programmatic advertising, survey programming, or statistical coding.
 - (e) Non-empirical publications such as editorials, position essays, or non-educational contexts.
3. Special Cases:
- (a) Robotics is treated as a domain-specific approach; studies were included only if programming was involved.
 - (b) AR/VR was included if AR/VR was the object of programming.
 - (c) Parametric modeling/CAD was included if there was script/graph programming (textual or visual).
 - (d) Music/creative media was included if learners wrote code/patches (e.g., creative coding).
 - (e) Block-based programming is treated as a contextual category and included if it meets the programming activity requirements. Its role is primarily as a bridge to text-based ecosystems (Resnick et al., 2009).

3.4 Data Extraction and Codebook

This search strategy, which utilized the ERIC and Scopus databases, resulted in 24 articles eligible for synthesis. However, we found that none of the articles from ERIC were included in the synthesis. The scope of extraction was limited to the following four core variables:

1. Programming Language includes Core (Python, C++, Julia), Contextual (including Java, R, MATLAB), Contextual-Block-based (Scratch, Snap!, Blockly), and Other/Emerging (C#, Kotlin, Swift, Dart/Flutter, JavaScript/TypeScript, Rust, Go, Scala).
2. Learning Approach includes Core (PjBL, PBL, IBL), Contextual (STEM/STEAM, Game-Based Learning, Domain-Specific Applications), and Other/Emerging (Creative Coding, Maker Education, Hybrid AI-Lab).
3. AI Readiness Label (AI label). Assessment was conducted by assigning a score of 1 for each item containing the following task indicators:
 - (a) Algorithm reasoning. Intuitively, this algorithm always appears if the given task involves creativity.
 - (b) Data structures, computation, or numerical packages (e.g., NumPy/MATLAB). This includes manually defining data according to the context being studied. In block-based environments (e.g., Scratch), simple variables or built-in blocks are not automatically counted as data structures, unless the author explicitly reports the practice of manipulating collections, lists, or equivalent data work from the given task.
 - (c) Data-centric elements like statistics, probability, or preprocessing. This includes data processing to generate visualizations or simulations.
 - (d) Problem-solving for errors in code, algorithms, and debugging. Intuitively, this problem-solving appears if the programming orientation is product-based.

If a task was oriented towards machine learning, it was given a score of 2. Based on this description, the maximum possible score is 6. A total score of 3 or less was categorized as "weak", and a score greater than 3 was categorized as "strong".

4. Creativity Characteristics were tabulated based on explicit terms appearing in the articles. The classification of creativity follows the dimensions from [Hu and Adey \(2002\)](#): Creative Process, characterized by creative thinking or imagination; Creative Product, consisting of knowledge, phenomena, technical products, or problem-solving; and Creative Trait, by measuring indicators such as fluency, flexibility, originality, or others. Articles that do not include a classification of creativity dimensions or exhibit general creativity characteristics are categorized by their domain-specific context of creativity.

3.5 Synthesis and Analysis Strategy

Synthesis was conducted descriptively and narratively on the four core extracted variables: programming language category (PLC), learning approach category (LAC), AI readiness label (AL), and creativity dimension (CD). The unit of analysis is the number of studies. Relational pathways are expressed through a combinatorial relationship α_{ij}^{ij} , where i and j refer to the i-th and j-th variables. Superscripts are assigned for PLC and LAC, while subscripts are assigned for AL and CD. In this context, we assume $\alpha_{ij} = \alpha_{ij}$. The relational pathway from PLC is α^1 , from LAC is α^2 , from AL is α_1 , and from CD is α_2 . The primary summary includes the frequency and proportion of PLC occurrence, the distribution of LAC, and the calculated strength of AL and CD. To map the relationships, a tabulation of PLC by category was first created to map the strength of AL α_1^1 . Second, a tabulation of LAC was compiled to view the CD from the programming tasks performed α_2^2 . Third, a cross-tabulation of PL and LA categories α^{12} was constructed. This cross-analysis was conducted in three parts: how this combination maps AL α_1^{12} , creativity dimensions α_2^{12} , and both simultaneously α_{12}^{12} . Each variable has the same total amount, and the numbers shown in the graph refer to the variables indicated by the arrows (Figure 1). The analysis in this study will focus on the black arrows. Meanwhile, the blue dashed arrows serve only as indicators of the original items. The red dashed arrows are not included in the analysis because they are identical to the black arrows with the same index.

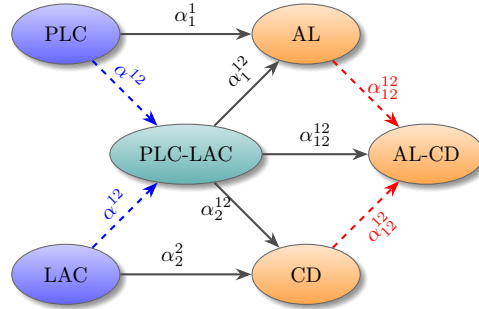


Fig. 1 The pathways of relationships among variables that can be synthesized

3.6 PRISMA Report

Reporting follows the PRISMA 2020 structure, comprising the identification, screening, and inclusion stages. Records from Scopus and ERIC were combined, deduplicated, and then screened by title and abstract before a full-text review. The numbers of articles at each stage are reported in Figure 2.

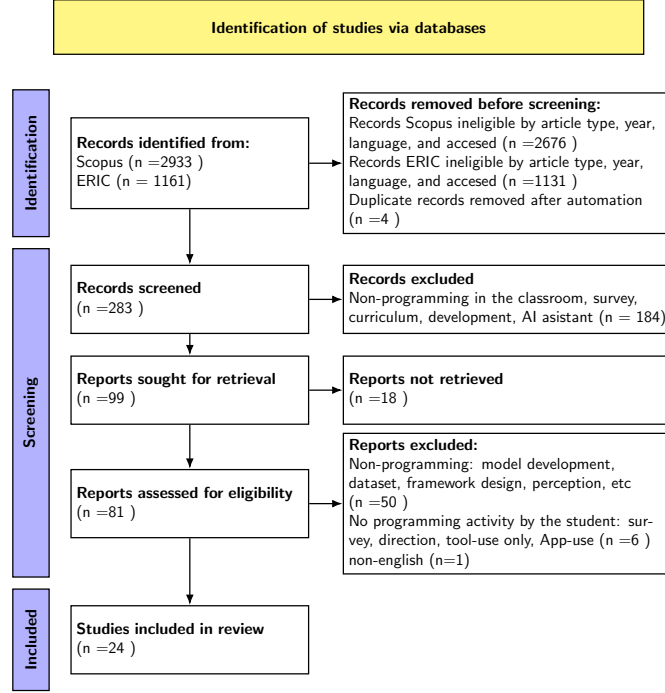


Fig. 2 PRISMA statement in Proggaming language and learning approach

4 Results

The synthesis is based on the 24 studies that met the inclusion criteria. The complete data tabulation related to the synthesis can be found in Table A1. The findings are presented below, organized according to the research questions and the established classification framework.

4.1 RQ1: Programming languages used in educational contexts to support AI readiness

Figure 3 presents a summary of the programming languages (PL) reported in the 24 included studies. The classification follows the programming language category

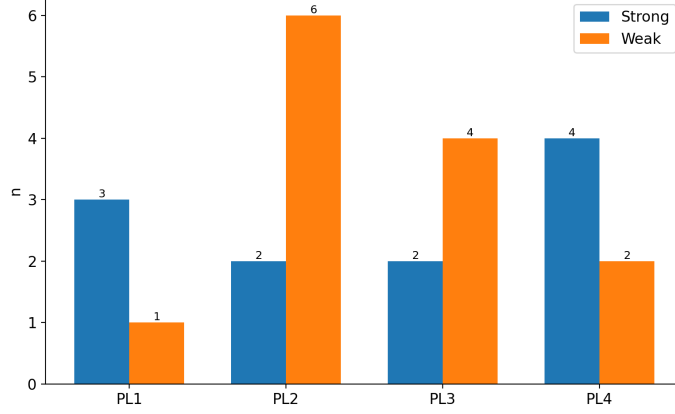


Fig. 3 AL distribution and strength per PLC

(PLC): core (PL1), contextual (PL2), contextual-block-based (PL3), and other/e-emerging (PL4), with the unit of analysis being the individual following the α_1^1 pathway in Figure 1. This tabulation serves as the basis for highlighting the proportion of each PLC to the assigned AI label (AL). For the purpose of this analysis, a "strong" label was given to studies that met more than three of the established computational criteria, such as the algorithm, the practice of data structure manipulation, processing data, or problem-solving, which were generally achieved through programming activities.

In the PL1, all findings originate from Python implementations with varying task characteristics. Two studies labeled as "strong" utilized medium- to long-term open-ended projects: a 10-week CS1 course that emphasized code explanation, testing, and decomposition, and a cross-domain creative project. Furthermore, a semester-long online course with a structured activity design that maintained repetitive practice confirms that duration strengthens the AL (Vadaparty et al. 2024; Hsueh et al. 2022). One "weak" implementation, which took place over three sessions in a secondary school robotics context, showed that a focus on device introduction and perception meant the accumulation of computational practice was not documented Rousouliotis et al. (2024). This comparison affirms that in PL1, the strength of the outcome is not determined by "Python" alone but by the task design: open-ended cross-domain projects, an emphasis on explanation-testing-decomposition, and sufficient duration for iteration and the accumulation of a computational trace.

In the PL2, studies labeled as "strong" emerged when computational representation became a core mechanism over a medium- to long-term horizon. Examples include the application of C/C++ (Arduino) over eleven weeks, which required the formulation of robot behavior and iterative testing, and a semester-long parametric modeling project with VR/3D printing artifacts as outputs (Sari et al. 2022; Hsu and Ou 2022). Short-duration implementations, such as a single session of GeoGebra commands in primary school and tangible coding (Cubetto), were labeled "weak" because evidence of accumulated computational practice was absent (Yunianto et al. 2025; Murcia et al. 2025).

In the PL3, "strong" achievement was observed when Scratch projects ran for approximately 12 weeks with explicit assessment of the creative product: the manipulation of control devices yielded different creativity profiles (fluency, flexibility, originality, elaboration), and top-down scaffolding improved the completeness and exploration of features (Chen et al. 2024; Su et al. 2024). "Weak" implementations were dominated by 1-3 session exposures (a one-day MakeCode course; teacher-student/pair programming interactions) or curricula that sharpened partial Computational Thinking (CT) components without artifact assessment (Olsson and Granberg 2024; Fagerlund et al. 2022; Smit et al. 2025; Cafarella and Vasconcelos 2025; Stein et al. 2025; Choleva et al. 2022).

In the PL4, strength was determined by task design and duration, not the language type itself: Processing/creative coding in an architecture studio, a robotics project based on HTML-CSS-JavaScript with a design thinking framework, a Thymio robot with delayed feedback, and a semester-long Game-Creative Learning course at the undergraduate level all showed clear patterns of artifact iteration (Domínguez-Gómez and Celis 2024; Díaz-Lauzurica and Moreno-Salinas 2023; Chevalier et al. 2022; Pietriková et al. 2025). Meanwhile, 1-3 session exposures based on PHP or Sonic Pi were labeled "weak" because their primary goal was experience or expression (Thongkoo et al. 2024; Cardoso et al. 2023). For a co-coding study with a generative model, the reflection-revision activities of Jonsson and Tholander (2022) produced expressive outputs even without a specified ecosystem.

Within this corpus, "strong" labels most frequently appeared when learning was medium- to long-term and product-oriented, with explicit artifact assessment. By educational level, the concentration of "strong" achievement is in higher education: Python is strong in introductory/online project-based courses running from 10 weeks to a full semester (Vadaparty et al. 2024; Hsueh et al. 2022). A similar pattern appears in domain-specific contexts at universities, such as Arduino robotics for pre-service teachers/undergraduates and a semester-long parametric design studio, which compel a formulate-test-revise cycle, thus resulting in a "strong" category (Sari et al. 2022; Hsu and Ou 2022). In primary education, "strong" achievement emerged mainly when Scratch projects lasted for multiple weeks and product creativity was assessed with a rubric, rather than being just a brief experience (Chen et al. 2024; Su et al. 2024). Other pathways strengthened when the programming medium truly became a means of artifact expression over a sufficient duration: creative coding in an undergraduate architecture studio, a web stack in a vocational project framed by design thinking, and C in a semester-long game-creative learning framework (Domínguez-Gómez and Celis 2024; Díaz-Lauzurica and Moreno-Salinas 2023; Pietriková et al. 2025). One important exception is an intense pedagogical intervention, such as delayed feedback on a Thymio robot, can produce "strong" achievement despite a short duration, because the quality of scaffolding compensated for the lack of accumulated time (Chevalier et al. 2022).

Generally, "weak" labels appeared when the exposure was very short and the main objective was initial experience, attitudes, or self-efficacy rather than the development of an explicitly assessed artifact. This pattern was visible in one-day or few-session visual/block-based tasks in primary and early secondary education, such as a daily MakeCode course emphasizing enjoyment and self-belief, a paired Scratch session

exploring the CT process, or algorithmic music/tangible robotics workshops oriented towards expression or concept introduction (Smit et al. 2025; Fagerlund et al. 2022; Cardoso et al. 2023; Murcia et al. 2025). "Weak" labels also appeared when short exposure occurred at higher levels, generally because the activity design was more centered on the device or learning setting than on the accumulation of computational practice. Examples include a three-session Python+Blockly robotics context that primarily measured motivation and perception, and a three-session IBL-based programming course at the undergraduate level that focused on ubiquitous learning management and collaboration rather than on traces of code iteration and artifact quality (Rousouliotis et al. 2024; Thongkoo et al. 2024).

Overall, "strong" labels are concentrated in medium to long-duration and product-oriented interventions. At the higher education level, this "strong" label is dominated by Python. At lower education levels, it is dominated by Scratch projects. "Weak" labels appear mainly in short duration interventions or when the measured indicators focus on attitudes or partial CT components instead of artifact quality. This pattern shows that duration and assessment design are more decisive than the categorization of the programming language alone.

4.2 RQ2: Learning approaches in programming are applied that characterize creativity

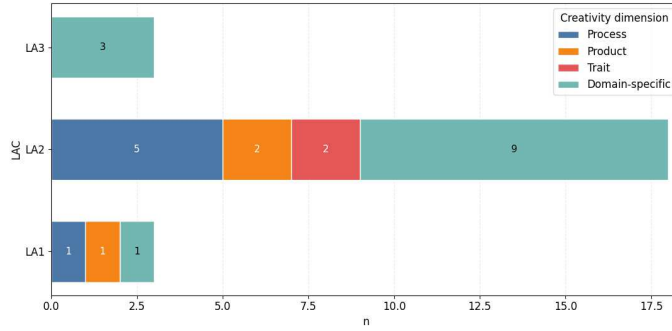


Fig. 4 The distribution of CD according to LAC

The following analysis focuses on the learning approaches (LA) that accompany programming practices. The learning approaches category (LAC) are grouped into three clusters: core (LA1), consisting of PjBL, PBL, and IBL; contextual (LA2), consisting of STEM/STEAM, game-based, and domain-specific applications; and other/emerging (LA3), consisting of creative coding, maker education, and hybrid/online models. These findings are shown in Figure 4 and refer to the α_2^2 pathway as illustrated in Figure 1. The strength of the findings is examined through the dimensions of creativity (CD), namely process, product, trait, and domain-specific.

In the LA1, the pattern of creativity's emergence follows the epistemic logic of each approach. In PjBL, the creative process dimension was apparent as students

gradually built a game by planning, collaborating, remixing/iterating, and revising solutions throughout the project. The evidence presented was in the format of work traces, namely class observations, worksheets, and reflections. This shows how creativity was enacted in that classroom. In PBL, domain-specific expression, which serves as an initial strategy to explore creativity, emerged, particularly creative mathematical reasoning. Analysis of teacher-student interactions and screen recordings revealed moments of proposing alternative strategies, generalization, and revision, which are creative qualities inherent to the practice of mathematizing while programming. Meanwhile, IBL tended to manifest creative products, namely the achieved solutions/code generated through collaborative investigation, discussion, and instructor feedback. Performance reports and task artifacts served as the primary evidence, meaning creativity was interpreted from the success in creating functional solutions and workable problem-solving.

In the LA2, CD appeared comprehensively. A dominance was shown in domain-specific contexts like robotics, mathematics, and music. In educational robotics based on the Thymio robot, creativity appeared as patterns of anticipation, formulation, and testing of the robot’s behavior, driven by a delayed feedback strategy, which characterizes a creative competence specific to the Educational Robotics (ER) domain. The process dimension was also dominant in this category in the form of creative thinking, which is naturally characterized by the STEM field. Creative products also appeared in this section in the form of problem-solving, such as in a GeoGebra activity that required artifact construction. Meanwhile, creative traits appeared in studies by [Su et al. \(2024\)](#) and [Chen et al. \(2024\)](#), manifested in concrete indicators that can quantitatively measure creativity and can be used as a reference to see the extent of creativity improvement in a learning classroom. In the LA3, only creativity related to the domain-specific dimension was reported, implying these studies have not yet addressed creativity in-depth through more detailed dimensions, while still highlighting general creativity within their respective fields.

In summary, this analysis reveals a clear pattern: while all learning approaches foster creativity in some form, the LA2 provides the richest and most diverse ecosystem for its expression and assessment. This cluster not only shows the highest frequency of reported creative outcomes but is also the only category where creativity is systematically operationalized and measured through concrete trait indicators. In contrast, the LA1 effectively highlights creativity within their specific epistemic frames, such as process in PjBL or product in IBL, while the LA3 shows potential but currently reports creativity in more general, domain-specific terms. This dominance of the contextual approach suggests that linking programming tasks to relevant, tangible applications (like STEM, robotics, or game design) creates a pedagogical environment where the multifaceted nature of creativity can be most fully observed, practiced, and, most importantly, formally assessed.

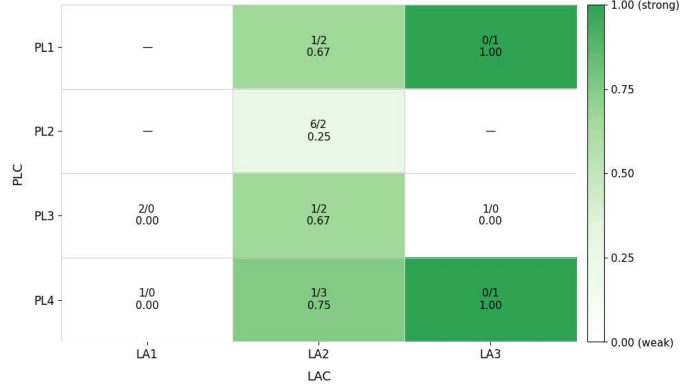


Fig. 5 The proportion of AL with respect to PLC and LAC

4.3 RQ3: The link between AI readiness and enhancing creativity supported by programming languages and learning approaches

This subsection prepares three synthesis lenses for RQ3. First, the AL is used as a weighted summary of each study's findings, following the pathway α_1^{12} based on the intersection of PL and LA. Second, the CD, as interpreted from the studies' reporting, follows the pathway α_2^{12} . Third, both AL and CD are analyzed following the pathway α_{12}^{12} .

Figure 5 visualizes the proportion of AL at each intersection of PLC and LAC within the α_1^{12} pathway. A value such as (1/3) would reflect 1 study with a "weak" AL and 3 studies with a "strong" AL, while the number below the label reflects the strength by dividing the value of the "strong" label (e.g., 3) by the sum of the weak and strong labels (e.g., 4). The figure reveals that the path to solid AL is not the result of a single combination, but rather the outcome of intentional pedagogical design. This map highlights that factors such as intervention duration, task depth, and a focus on higher-order computational skills are more influential than the mere choice of language or approach label.

The most significant finding is the superiority of the LA3 approach when applied at the higher education level, which consistently yields 100% "strong" labels. In the core language combination (PL1 $\times$ LA3), the study by [Vadaparty et al. \(2024\)](#) demonstrates how integrating LLMs into an introductory Python course shifts the focus from writing syntax to essential skills like problem decomposition, testing, and code explanation. Similarly, in the other languages domain (PL4 $\times$ LA3), the study by [Jonsson and Tholander \(2022\)](#) on co-coding with AI in creative programming found that the "friction" from the AI's imperfect results could spark reflection and the exploration of alternative solutions. Both of these studies, which can be categorized as hybrid AI-lab classrooms, underscore that strong AI readiness is built through reflective, authentic problem-solving-oriented computational practices, in line with the principle of literate computing.

On the other hand, LA2, which is the most common category in the study corpus, shows highly varied results. This indicates that the "contextual" label alone does not guarantee the achievement of strong AI readiness; duration and implementation complexity are the determining factors. Studies labeled "strong" in this category consistently involve long-term, product-oriented interventions. Examples include an eleven-week, STEM-based Arduino robotics activity that demanded a cycle of formulation and iterative testing by [Sari et al. \(2022\)](#), and a semester-long parametric modeling project in an architecture studio by [Hsu and Ou \(2022\)](#). Conversely, studies labeled "weak" are generally short-term exposures aimed at concept introduction and motivation, such as a single session with GeoGebra by [Yunianto et al. \(2025\)](#) or tangible coding with Cubetto in early childhood education by [Murcia et al. \(2025\)](#).

Meanwhile, the low proportion of "strong" labels in LA1, such as PBL and IBL, requires careful interpretation. This finding does not necessarily conclude that these approaches are ineffective. Rather, it reflects the research focus of the reviewed studies. For example, the IBL study by [Thongkoo et al. \(2024\)](#) focused more on the collaborative learning management aspect than on the technical complexity of the resulting programming artifacts. Similarly, the study by [Olsson and Granberg \(2024\)](#) used a problem-based framework to analyze teacher-student interactions in supporting mathematical reasoning, not to measure the depth of computational practice. Thus, the "weak" performance in the LA1 cluster is likely an artifact of the misalignment between the research focus of those studies and the operational criteria for the AI readiness label used in this review.

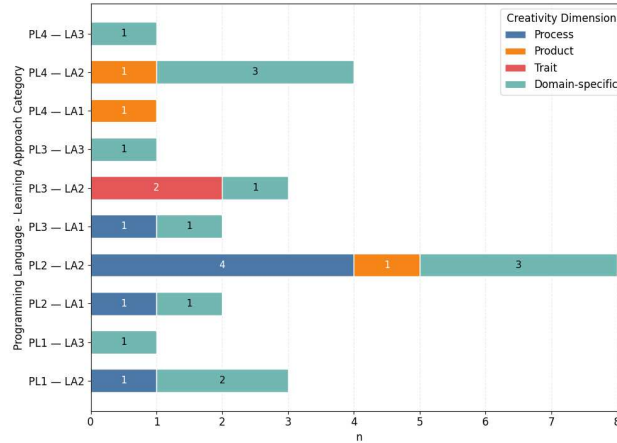


Fig. 6 Distribution of CD based on PLC and LAC

Figure 6 maps the distribution of CD through the α_2^{12} pathway. Its interpretation must be viewed through a hierarchical lens that reflects how creativity is conceptualized and measured. Based on guidance from the OECD (2022), every form of

creativity is fundamentally domain-specific; it is always bound to a specific context or field of knowledge. Within this framework, creativity can be observed through "Process" (e.g., ways of thinking or imagining) and manifested in a "Product" (e.g., an artifact or a problem solution). However, the most granular and concretely measurable level of analysis lies in the "Trait" dimension, which uses explicit indicators such as fluency, flexibility, and originality to quantify the creativity manifested in the process or product. With this understanding, Figure 6 shows that the majority of the literature in this corpus reports creativity at a descriptive level, namely by identifying the presence of a creative process or product within a specific domain. However, only a small fraction of studies go further to perform formal measurement. The point where this measurement occurs most frequently is in the PL2 $\times$ LA2 combination. The studies by [Su et al. \(2024\)](#) and [Chen et al. \(2024\)](#) are perfect examples of this hierarchy. Both investigated creativity in the domain of game creation with Scratch, which was manifested in a student-created game product, and was rigorously measured using creativity trait indicators. This makes their findings particularly strong because they not only describe but also systematically measure the creative outcomes. Meanwhile, other combinations more frequently report creativity at a descriptive level. The most prominent combination by frequency, PL2 languages with LA2, turns out to be the largest contributor to reports on creativity from the "Process" dimension. This indicates that in the context of PL2 languages like Java, R, or MATLAB in projects, researchers tend to focus on describing the workflow, collaboration, and problem-solving strategies as manifestations of creativity, rather than measuring it with trait indicators. This descriptive pattern also applies to other programming languages. The use of Python in PL1 is reported to trigger creativity that is manifested in "Process" and "Domain-specific" solutions, where the novelty of the solution in a domain like data science is the main emphasis. Similarly, the creativity of PL4 tends to be reported through the evaluation of the final "Product" and its relevance in a "Domain-specific" context. Overall, this updated data sharpens the conclusion. The majority of the literature reports creative engagement in programming education, but mostly in the form of qualitative descriptions of process and product within a specific domain. The practice of rigorous and systematic creativity measurement to date appears to be a phenomenon concentrated in the block-based programming ecosystem for learners in primary and secondary education. This underscores an urgent need to develop and apply equivalent trait-based assessment methods for the context of text-based programming instruction at higher levels. The cross-analysis between the AL map (Figure 5) and the CD map (Figure 6) within the α_{12}^{12} pathway reveals a complex and non-linear relationship between the two outcomes. This finding shows that the most effective pedagogical combinations for producing "strong" AI readiness are not always the same as those most effective for concretely measuring creativity. At least three main patterns emerge: (1) a synergistic nexus where both outcomes can be fostered and measured, (2) a divergence where a focus on deep AI readiness is accompanied by more general creativity reporting, and (3) an area rich in creative process but with weaker evidence of AI readiness. The clearest synergistic pattern is seen in the PL3 $\times$ LA2 combination. This combination is the only domain where creativity is formally measured using trait indicators. When we look at the AI readiness map for

the same combination, we find a reasonably high proportion of "strong" labels (0.67). This shows that a block-based learning environment implemented in a medium-term project not only provides fertile ground for fostering and rigorously measuring creativity but is also very effective in building a foundation for AI readiness. It can be concluded that for primary and secondary education levels, this approach represents an ideal "sweet spot" for achieving both learning goals simultaneously. An interesting divergence pattern emerges in the context of advanced learning. The combinations that produce the "strongest" AI readiness (a proportion of 1.00) are the core (Python) and other/emerging languages within an other/emerging approach (PL1 $\times$ LA3 and PL4 $\times$ LA3). However, when we look at the creativity map for these combinations, the reported creativity is limited to a "Domain-specific" description. There is no measurement of traits or even detailed descriptions of process and product. This indicates a shift in focus. At the university level with highly complex tasks, the focus of learning and research appears to shift to deep technical mastery and the ability to solve complex domain problems. Creativity here is expressed as an "elegant and innovative solution" and tends to be considered an inseparable part of technical expertise, and is thus no longer broken down and measured using more fundamental trait indicators. The third pattern is seen in the use of text-based contextual languages (like Java/R) in a contextual approach, PL2 $\times$ LA2. This combination, according to the creativity map, is the largest contributor to reports on creativity from the "Process" dimension. However, when viewed from the AI readiness map, this combination has a relatively low proportion of "strong" labels (0.25). This points to an area where the main focus is on the learning journey and the problem-solving process, how students think, collaborate, and design solutions. However, the computational artifacts produced may not yet have reached the complexity required to be categorized as "strong" within the AI readiness framework. Overall, this synthesis confirms that educators and curriculum designers need to make strategic choices. If the goal is to build a foundation of measurable creativity along with solid AI readiness in the early stages, then the PL3 $\times$ LA2 approach is the most promising. However, if the goal is to push the boundaries of technical mastery for AI readiness at an advanced level, the focus may shift to the LA3 approach, with the awareness that formal creativity measurement may no longer be a primary priority.

5 Discussion

5.1 Synthesis and Interpretation of Key Findings

This systematic review reveals that efforts to foster AI readiness and creativity through programming education are not determined by the choice of language or pedagogical approach label alone. Instead, the effectiveness of an intervention consistently depends on a deeper level of implementation design. This is associated with adequate duration, a product or project orientation, and a focus on reflective computational practices. The analysis of the 24 included studies shows a pattern of both synergy and divergence between the goals of fostering AI readiness and creativity. These patterns imply that optimal pedagogical strategies need to be carefully tailored to specific educational levels and learning objectives.

One of the most consistent findings from this analysis is that the duration and depth of student engagement are far more influential in building AI readiness than the abstract category of the learning approach. Interventions that take place over the medium to long term and require students to produce complex computational artifacts consistently yield a "strong" AI label (Hsu and Ou 2022; Sari et al. 2022; Vadaparty et al. 2024). Such a design forces students to go beyond syntactic understanding and engage in an authentic development cycle: problem decomposition, algorithm design, iterative testing, and debugging. These practices are the core of AI readiness. Conversely, short interventions, while often successful in increasing motivation or introducing initial concepts (Murcia et al. 2025; Yudianto et al. 2025), fail to provide sufficient time for students to accumulate deep computational practice, thus resulting in a "weak" AI label.

The cross-analysis highlights the combination of block-based programming in contextual projects (PL3 $\times$ LA2) as a sweet spot, especially for primary and secondary education levels. This combination is not only proven effective in building a foundation for AI readiness, but it is also the only domain in this review where creativity is formally and systematically measured using trait indicators (Chen et al. 2024; Su et al. 2024). This finding confirms that a platform like Scratch, when used in a structured and sufficiently long project, is not merely a playful introductory tool. Instead, it functions as a serious and mature pedagogical ecosystem capable of balancing the development of computational thinking and creative expression simultaneously without overburdening students with syntactic complexity.

A significant divergence is revealed at higher education levels. The pedagogical combinations proven most effective in producing a "strong" AI readiness, namely the use of Python or other emerging languages in a hybrid AI-lab approach (PL1/PL4 $\times$ LA3) (Jonsson and Tholander 2022; Vadaparty et al. 2024). Both are the very ones that report creativity in the most general and unmeasured way. This pattern indicates a shift in learning objective priorities. At advanced levels, the focus shifts from fostering creativity as a separate trait to applying creativity as an integral part of complex technical problem-solving. Here, creativity is expressed through the novelty and elegance of the technical solution itself, and success is measured by the quality and functionality of the final product, rather than through separate creativity indicators like fluency or flexibility.

5.2 Dialogue with Existing Literature

The findings of this systematic review do not stand alone; they resonate and interact with broader discussions in the fields of AI literacy, programming education, and creativity research. By placing the results of the analysis within the context of existing literature, we can see the deeper contributions and implications of this study.

Literature on AI literacy has proposed various conceptual frameworks outlining the competencies needed to critically understand and interact with AI systems (Long and Magerko 2020; Lee et al. 2021). These frameworks often include an understanding of core AI concepts, the ability to use them, and ethical awareness. The findings from this SLR provide empirical validation for these components from a pedagogical practice perspective. Studies labeled "strong," particularly at the higher education level,

consistently shift the focus from mere syntax instruction to higher-order skills such as problem decomposition, testing, and code justification (Vadaparty et al. 2024). These practices directly reflect and operationalize the computational thinking competencies that are the foundation of AI literacy, as advocated by Wing (2006) and further detailed by Shute et al. (2017). Thus, this SLR bridges the gap between theoretical AI literacy frameworks and the concrete instructional designs that have proven effective in the field.

The finding that duration and project orientation are stronger predictors of success than the name of the approach aligns with extensive evidence supporting the effectiveness of PjBL in enhancing engagement and deep understanding (Hmelo-Silver 2004; Kokotsaki et al. 2016). This SLR strengthens this principle by adding a crucial nuance: for the context of AI readiness, a project must not only be authentic but must also demand a significant accumulation of computational practice over time. Furthermore, this finding expands the role of block-based programming platforms. Classic literature often positions Scratch and similar tools as a "bridge" or scaffold towards "real" text-based programming (Resnick et al. 2009; Weintrop and Wilensky 2015). However, our analysis shows a more fundamental role. The block-based environment in this review emerges as the primary ecosystem where creativity is not only encouraged but is also explicitly measured using trait indicators. This indicates that such platforms are not merely transitional tools, but mature pedagogical environments with unique advantages in creativity assessment that are difficult to replicate in text-based contexts at the same level.

Finally, when these findings are juxtaposed with the creativity literature, a significant gap becomes clear. The standard definition of creativity requires both novelty and effectiveness (Runco and Jaeger 2012). While an elegant technical solution in an advanced programming project could be said to meet this criterion holistically (Jonsen and Tholander 2022), this SLR shows that the granular measurement of creativity traits (fluency, flexibility, originality) is almost never conducted in such contexts. The practice of formal and systematic creativity assessment appears to stop at the secondary education level. This implies that the field of advanced programming education, while implicitly valuing creativity, has not yet widely adopted or developed rigorous assessment methodologies to measure it. This gap is also acknowledged in the literature, where researchers highlight the challenges in objectively defining and measuring aspects of creativity in programming, such as the 'elegance' or 'efficiency' of a code solution (Lishinski et al. 2016). Consequently, the understanding of how technical creativity develops or how various pedagogical approaches affect it at an expert level remains limited.

5.3 Practical Implications

The findings of this systematic review offer significant practical guidance for stakeholders in the field of education. For educators and curriculum designers, the main implication is the need for a shift from isolated, short-term programming activities towards the design of more in-depth and authentic learning experiences. At the primary and secondary education levels, the most promising model is the implementation

of multi-week, project-based curricula using block-based platforms. Rather than simply completing small exercises, students should be encouraged to build meaningful digital artifacts, where assessment focuses not only on program functionality but also uses clear rubrics to evaluate the mastery of computational concepts and the creative aspects of the product in a balanced manner (Chen et al. 2024; Su et al. 2024). At the higher education and vocational levels, curricula must move beyond pure syntax instruction and shift towards complex, semester-long projects that mirror real-world challenges. Educators can adopt reflective pedagogies where students are required not only to write code but also to explain, test, and systematically decompose problems (Vadaparty et al. 2024). Furthermore, to foster creativity, programming should be positioned as a tool for cross-disciplinary innovation, for example, through the application of creative coding in art and design or parametric modeling in architecture (Domínguez-Gómez and Celis 2024; Hsu and Ou 2022).

These implications also extend to the policy level. Policymakers are encouraged to prioritize support for the depth and quality of curriculum implementation over mere brief exposure. Policies and funding should be directed to support the development of integrated, long-term curricula, not just sporadic events that, while good for introduction, are proven insufficient for building substantive skills. More crucially, the success of these more complex learning models is highly dependent on educator expertise. Therefore, a significant investment must be allocated for sustainable, high-quality teacher professional development programs. This training must equip teachers not only with technical skills but also with the pedagogical competencies to facilitate project-based learning, manage open-ended tasks, and conduct effective formative assessments, a strategy proven essential for successful implementation in the field (Sari et al. 2022).

5.4 Limitations

This study has several limitations. First, the scope of the literature search was limited to peer-reviewed articles in English with a DOI published between January 1, 2022, and August 14, 2025, which means this review represents a snapshot of recent, formal research. The findings may not capture innovative practices documented in other formats (e.g., technical reports, pre-prints, or books) or research conducted outside of this linguistic region. Second, there are methodological limitations inherent in the synthesis process. The classification of the AI readiness label into "strong" and "weak" is a proxy operationalized for the analytical purposes of this study. Although applied systematically based on clear criteria, this label is a simplification of a complex and multifaceted construct. Furthermore, the analysis of creativity dimensions is highly dependent on how the original authors reported and measured them; it is possible that richer creative activities occurred in the field than were explicitly reported in the publications. Finally, as with all literature reviews, this study is susceptible to potential publication bias, where studies with positive or statistically significant results are more likely to be published, potentially presenting a more optimistic picture than the reality in the field.

5.5 Future Research Directions

Based on the findings and limitations outlined, this review highlights several promising directions for future research. First, there is an urgent need to develop and validate assessment instruments that can rigorously measure creativity in the context of advanced, text-based programming. As our findings show a void in the measurement of creativity traits at this level, future research could focus on designing rubrics or metrics capable of capturing not only product functionality but also aspects such as the elegance, efficiency, and originality of algorithmic solutions, a challenge also acknowledged in the literature (Lishinski et al. 2016). Second, to move beyond the correlational findings of this review, more comparative studies with experimental or quasi-experimental designs are needed. For example, a study could directly compare the effectiveness of a contextual approach (LA2) versus a hybrid AI-lab approach (LA3) in teaching Python to observe their controlled impact on AI readiness and creativity. Finally, to understand how these skills develop over time, longitudinal research is essential. Tracking the development of AI readiness and creativity in a cohort of students over several years would provide invaluable insight into how early interventions affect later success and how the transition from block-based to text-based environments occurs effectively.

6 Conclusion

This systematic literature review was conducted to map and synthesize current evidence regarding the programming languages and pedagogical approaches used to support AI readiness while enhancing creativity in educational contexts. The analysis of 24 relevant studies shows that the effectiveness of a programming language is not inherent but is critically determined by the design of its pedagogical implementation. Key factors such as a sufficiently long intervention duration, an authentic project or product orientation, and a focus on reflective computational practices proved to be stronger predictors of success than the mere choice of tool. Furthermore, this study identified a complex relationship between AI readiness and creativity. A synergistic nexus was found in contextual, block-based learning environments at the primary education level, where both outcomes can be fostered and measured simultaneously. However, at higher educational levels, a divergence occurs where a focus on achieving deep AI readiness is often accompanied by more holistic and less measured reporting of creativity. The main contribution of this review is the provision of a two-dimensional framework for classifying languages and approaches, which serves as a strategic map for educators and curriculum designers. This framework helps them make more informed decisions in aligning tools and methods with specific learning goals. The goals are to optimize for measurable creativity, deep technical readiness, or a balance between the two, in order to more effectively prepare students for an AI-driven future.

Appendix A The articles included in the synthesis

The data of the synthesized articles are presented in Table A1. These data include the authors' names, programming language category (PLC), programming languages used (PL), learning approach category (LAC), learning approaches (LA), creativity dimension category (CKC), and AI label. Additional information on education level and learning duration is also provided to support the analysis.

References

- Abadi M, Barham P, Chen J, et al (2016) TensorFlow: A system for large-scale machine learning. <https://doi.org/10.48550/arXiv.1605.08695>, URL <http://arxiv.org/abs/1605.08695>, arXiv:1605.08695 [cs]
- Adetiba E, Adeyemi-Kayode TM, Akinrinmade AA, et al (2021) Evolution of Artificial Intelligence Programming Languages - a Systematic Literature Review. *Journal of Computer Science* 17(11):1157–1171. <https://doi.org/10.3844/jcssp.2021.1157.1171>, URL <https://thescipub.com/abstract/jcssp.2021.1157.1171>
- Benitti FBV (2012) Exploring the educational potential of robotics in schools: A systematic review. *Computers & Education* 58(3):978–988. <https://doi.org/10.1016/j.compedu.2011.10.006>, URL <https://www.sciencedirect.com/science/article/pii/S0360131511002508>
- Bezanson J, Edelman A, Karpinski S, et al (2017) Julia: A Fresh Approach to Numerical Computing. *SIAM Review* 59(1):65–98. <https://doi.org/10.1137/141000671>, URL <https://epubs.siam.org/doi/10.1137/141000671>, publisher: Society for Industrial and Applied Mathematics
- Blake-West J, Alrawashdeh G, Bers M (2024) Validating a Creative Coding Rubric through expressive activities for elementary grades. *Journal of Research on Technology in Education* 0(0):1–20. <https://doi.org/10.1080/15391523.2024.2398502>, URL <https://doi.org/10.1080/15391523.2024.2398502>, publisher: Routledge _eprint: <https://doi.org/10.1080/15391523.2024.2398502>
- Bond M, Bedenlier S, Marín VI, et al (2021) Emergency remote teaching in higher education: mapping the first global online semester. *International Journal of Educational Technology in Higher Education* 18(1):50. <https://doi.org/10.1186/s41239-021-00282-x>, URL <https://doi.org/10.1186/s41239-021-00282-x>
- Cafarella L, Vasconcelos L (2025) Computational thinking with game design: An action research study with middle school students. *Education and Information Technologies* 30(5):5589–5633. <https://doi.org/10.1007/s10639-024-13010-5>, URL <https://doi.org/10.1007/s10639-024-13010-5>
- Cardoso MA, Morgado EMG, Leonido L (2023) Unleashing Creative Synergies: A Mixed-Method Case Study in Music Education Classrooms. *Applied Sciences*

Table A1 Summary of studies categorized by PLC, programming language, learning approach, CKC, AI label, education level, and duration.

No	Author	PLC	PL	LAC	LA	CKC	AI Label	Education Level	Duration
1	Smit et al. (2025)	PL2	Block-Base MakeCode	LA2	STEM	DS	Weak	Upper Secondary	1 session
2	Cafarella and Vasconcelos (2025)	PL2	Block-base code.org	LA2	Game-based	DS	Weak	Upper Secondary	9 weeks
3	Stein et al. (2025)	PL2	Block-Base NetsBlox	LA2	Domain-specific robotic (simulation)	DS	Weak	Non formal	NR
4	Sari et al. (2022)	PL2	C/C++ (Arduino)	LA2	Domain-Specific Robotic	Process	Strong	Undergraduate	11 weeks
5	Hsu and Ou (2022)	PL2	Parametric Modeling	LA2	Parametric Design	Process	Strong	Undergraduate	1 Semester
6	Murcia et al. (2025)	PL2	Cubetto	LA2	STEM	Process	Weak	Lower	1 session
7	Liu et al. (2022)	PL2	Block-base	LA2	Domain-Specific Robotic	Process	Weak	Upper Secondary	6 weeks
8	Yunianto et al. (2025)	PL2	GeoGebra scripting/-commands	LA2	Domain-specific Mathematics	Product	Weak	Primary School	1 session
9	Ye et al. (2024)	PL3	Block-Base Scratch	LA2	Programming-Based	DS	Weak	Undergraduate	5 weeks
10	Su et al. (2024)	PL3	Block-Base Scratch	LA2	Programming-Based	Trait	Strong	Primary School	12 weeks
11	Chen et al. (2024)	PL3	Block-Base Scratch	LA2	Game-based	Trait	Strong	Primary School	12 weeks
12	Domínguez-Gómez and Celis (2024)	PL1	Python	LA2	Programming-Based	DS	Strong	Undergraduate	5 sessions
13	Rousouliotis et al. (2024)	PL1	Python	LA2	Domain-Specific Robotic	DS	Weak	Secondary and High School	3 sessions
14	Hsueh et al. (2022)	PL1	Python	LA2	Hybrid/Online	Process	Strong	Undergraduate	1 Semester
15	Chevalier et al. (2022)	PL4	Thymio	LA2	Domain-Specific CCPS	DS	Strong	8–9 years old (online)	45 minutes
16	Pietriková et al. (2025)	PL4	C	LA2	Game-based	DS	Strong	Undergraduate	1 Semester
17	Cardoso et al. (2023)	PL4	Sonic Pi (Ruby-based)	LA2	Domain-Specific Music	DS	Weak	10–12 years old (music school)	3 sessions
18	Díaz-Lauzurica and Moreno-Salinas (2023)	PL4	JavaScript, HTML, CSS	LA2	Domain-Specific Robotic	Product	Strong	Undergraduate	1–2 months
19	Olsson and Granberg (2024)	PL3	Block-Base Scratch	LA1	PBL	DS	Weak	Primary School	2 sessions
20	Choleva et al. (2022)	PL3	Block-Base Scratch	LA1	PjBL	Process	Weak	Primary School	12 weeks
21	Thongkoo et al. (2024)	PL4	PHP	LA1	IBL	Product	Weak	Undergraduate	3 sessions
22	Fagerlund et al. (2022)	PL3	Block-Base Scratch	LA3	Coding Design	DS	Weak	Primary School	1 session
23	Vadaparty et al. (2024)	PL1	Python	LA3	Course-Context	DS	Strong	Undergraduate	10 weeks
24	Jonsson and Tholander (2022)	PL4	JavaScript	LA3	Creative Programming	DS	Strong	Undergraduate	6 weeks

- 13(17):9842. <https://doi.org/10.3390/app13179842>, URL <https://www.mdpi.com/2076-3417/13/17/9842>, publisher: Multidisciplinary Digital Publishing Institute
- Chen CY, Su SW, Yuan SM (2024) The Effect of Regular and Innovative Control Devices on Cultivating Creativity in a Game Creating Course in Primary School. *Education Sciences* 14(8):833. <https://doi.org/10.3390/educsci14080833>, URL <https://www.mdpi.com/2227-7102/14/8/833>, publisher: Multidisciplinary Digital Publishing Institute
- Chen E, Asta M (2022) Using Jupyter Tools to Design an Interactive Textbook to Guide Undergraduate Research in Materials Informatics. *Journal of Chemical Education* 99(10):3601–3606. <https://doi.org/10.1021/acs.jchemed.2c00640>, URL <https://pubs.acs.org/doi/10.1021/acs.jchemed.2c00640>
- Chen T, Guestrin C (2016) XGBoost: A Scalable Tree Boosting System. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, New York, NY, USA, KDD '16, pp 785–794, <https://doi.org/10.1145/2939672.2939785>, URL <https://dl.acm.org/doi/10.1145/2939672.2939785>
- Chevalier M, Giang C, El-Hamamsy L, et al (2022) The role of feedback and guidance as intervention methods to foster computational thinking in educational robotics learning activities for primary school. *Computers & Education* 180:104431. <https://doi.org/10.1016/j.compedu.2022.104431>, URL <https://www.sciencedirect.com/science/article/pii/S0360131522000021>
- Choleva V, Stagakis S, Choleva E, et al (2022) Integrating the 4cs into Creating Games by Visual Programming and Project Based Learning. *European Conference on Games Based Learning* 16(1):147–154. <https://doi.org/10.34190/ecgbl.16.1.694>, URL <https://papers.academic-conferences.org/index.php/ecgbl/article/view/694>
- Cooper S, Dann W, Pausch R (2003) Teaching objects-first in introductory computer science. *SIGCSE Bull* 35(1):191–195. <https://doi.org/10.1145/792548.611966>, URL <https://doi.org/10.1145/792548.611966>
- Díaz-Lauzurica B, Moreno-Salinas D (2023) Applying Design Thinking to Enhance Programming Education in Vocational and Compulsory Secondary Schools. *Applied Sciences* 13(23):12792. <https://doi.org/10.3390/app132312792>, URL <https://www.mdpi.com/2076-3417/13/23/12792>, publisher: Multidisciplinary Digital Publishing Institute
- Domínguez-Gómez P, Celis F (2024) Creative programming in architecture: a computational thinking approach. *Informatics in Education* 23(3):541–570. <https://doi.org/10.15388/infedu.2024.18>, URL <https://infedu.vu.lt/journal/INFEDU/article/781>, publisher: Vilnius University Institute of Data Science and Digital Technologies

- Donovan AAA (2016) The Go programming language. Addison-Wesley professional computing series, Addison-Wesley, New York
- Fagerlund J, Vesisenaho M, Häkkinen P (2022) Fourth grade students' computational thinking in pair programming with Scratch: A holistic case analysis. *International Journal of Child-Computer Interaction* 33:100511. <https://doi.org/10.1016/j.ijcci.2022.100511>, URL <https://www.sciencedirect.com/science/article/pii/S2212868922000381>
- Friedman JH, Hastie T, Tibshirani R (2010) Regularization Paths for Generalized Linear Models via Coordinate Descent. *Journal of Statistical Software* 33:1–22. <https://doi.org/10.18637/jss.v033.i01>, URL <https://doi.org/10.18637/jss.v033.i01>
- Haleem A, Javaid M, Qadri MA, et al (2022) Understanding the role of digital technologies in education: A review. *Sustainable Operations and Computers* 3:275–285. <https://doi.org/10.1016/j.susoc.2022.05.004>, URL <https://www.sciencedirect.com/science/article/pii/S2666412722000137>
- Hall M, Frank E, Holmes G, et al (2009) The WEKA data mining software: an update. *SIGKDD Explor Newsl* 11(1):10–18. <https://doi.org/10.1145/1656274.1656278>, URL <https://doi.org/10.1145/1656274.1656278>
- Harris CR, Millman KJ, van der Walt SJ, et al (2020) Array programming with NumPy. *Nature* 585(7825):357–362. <https://doi.org/10.1038/s41586-020-2649-2>, URL <https://www.nature.com/articles/s41586-020-2649-2>, publisher: Nature Publishing Group
- Higgins JPT, López-López JA, Becker BJ, et al (2019) Synthesising quantitative evidence in systematic reviews of complex health interventions. *BMJ global health* 4(Suppl 1):e000858. <https://doi.org/10.1136/bmjgh-2018-000858>
- Higham DJ, Higham NJ (2017) MATLAB guide, third edition edn. No. 150 in Other titles in applied mathematics, siam, Society for Industrial and Applied Mathematics, Philadelphia
- Hmelo-Silver CE (2004) Problem-Based Learning: What and How Do Students Learn? *Educational Psychology Review* 16(3):235–266. <https://doi.org/10.1023/B:EDPR.0000034022.16470.f3>, URL <https://doi.org/10.1023/B:EDPR.0000034022.16470.f3>
- Hsu CY, Ou SJ (2022) Innovative Practice of Sustainable Landscape Architecture Education—Parametric-Aided Design and Application. *Sustainability* 14(8):4627. <https://doi.org/10.3390/su14084627>, URL <https://www.mdpi.com/2071-1050/14/8/4627>, publisher: Multidisciplinary Digital Publishing Institute
- Hsueh NL, src='/Content/Images/orcid.gif' /> BDhocl, Lai LC (2022) Exploring the Influence of Students' Modes of Behavioral Engagement in an Online

- Programming Course Using the Partial Least Squares Structural Equation Modeling Approach. *Journal of Information Technology Education: Research* 21:403–423. URL <https://www.informingscience.org/Publications/5010>
- Hu W, Adey P (2002) A scientific creativity test for secondary school students. *International Journal of Science Education* 24(4):389–403. <https://doi.org/10.1080/09500690110098912>, URL <https://doi.org/10.1080/09500690110098912>, publisher: Routledge _eprint: <https://doi.org/10.1080/09500690110098912>
- Innes M (2018) Flux: Elegant machine learning with Julia. *Journal of Open Source Software* 3(25):602. <https://doi.org/10.21105/joss.00602>, URL <https://joss.theoj.org/papers/10.21105/joss.00602>
- Jonsson M, Tholander J (2022) Cracking the code: Co-coding with AI in creative programming education. In: *Proceedings of the 14th Conference on Creativity and Cognition*. Association for Computing Machinery, New York, NY, USA, C&C '22, pp 5–14, <https://doi.org/10.1145/3527927.3532801>, URL <https://dl.acm.org/doi/10.1145/3527927.3532801>
- Klabnik S, Nichols C (2018) *The Rust Programming Language*. No Starck Press, San Francisco, URL <https://lccn.loc.gov/2018014097>
- Kluyver T, Ragan-Kelley B, Perez F, et al (2016) Jupyter Notebooks – a publishing format for reproducible computational workflows. In: *Positioning and Power in Academic Publishing: Players, Agents and Agendas*. IOS Press, <https://doi.org/10.3233/978-1-61499-649-1-87>, URL <https://www.medra.org/servelet/aliasResolver?alias=iospressISBN&isbn=978-1-61499-648-4&spage=87&doi=10.3233/978-1-61499-649-1-87>
- Kokotsaki D, Menzies V, Wiggins A (2016) Project-based learning: A review of the literature. *Improving Schools* 19(3):267–277. <https://doi.org/10.1177/1365480216659733>, URL <https://doi.org/10.1177/1365480216659733>, publisher: SAGE Publications Ltd
- Kuhn M (2008) Building Predictive Models in R Using the caret Package. *Journal of Statistical Software* 28:1–26. <https://doi.org/10.18637/jss.v028.i05>, URL <https://doi.org/10.18637/jss.v028.i05>
- Lazonder AW, Harmsen R (2016) Meta-Analysis of Inquiry-Based Learning: Effects of Guidance. *Review of Educational Research* 86(3):681–718. <https://doi.org/10.3102/0034654315627366>, URL <https://doi.org/10.3102/0034654315627366>, publisher: American Educational Research Association
- Lee I, Ali S, Zhang H, et al (2021) Developing Middle School Students' AI Literacy. In: *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education*. ACM, Virtual Event USA, pp 191–197, <https://doi.org/10.1145/3408877.3432513>, URL <https://dl.acm.org/doi/10.1145/3408877.3432513>

- Lefebvre C, Glanville J, Briscoe S, et al (2019) Searching for and selecting studies. In: *Cochrane Handbook for Systematic Reviews of Interventions*. John Wiley & Sons, Ltd, p 67–107, <https://doi.org/10.1002/9781119536604.ch4>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119536604.ch4>, section: 4 _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781119536604.ch4>
- Li X, Huang ZJ (2017) An inverted classroom approach to educate MATLAB in chemical process control. *Education for Chemical Engineers* 19:1–12. <https://doi.org/10.1016/j.ece.2016.08.001>, URL <https://www.sciencedirect.com/science/article/pii/S1749772816300252>
- Lishinski A, Yadav A, Enbody R, et al (2016) The Influence of Problem Solving Abilities on Students’ Performance on Different Assessment Tasks in CS1. In: *Proceedings of the 47th ACM Technical Symposium on Computing Science Education*. Association for Computing Machinery, New York, NY, USA, SIGCSE ’16, pp 329–334, <https://doi.org/10.1145/2839509.2844596>, URL <https://doi.org/10.1145/2839509.2844596>
- Liu H, Sheng J, Zhao L (2022) Innovation of Teaching Tools during Robot Programming Learning to Promote Middle School Students’ Critical Thinking. *Sustainability* 14(11):6625. <https://doi.org/10.3390/su14116625>, URL <https://www.mdpi.com/2071-1050/14/11/6625>, publisher: Multidisciplinary Digital Publishing Institute
- Long D, Magerko B (2020) What is AI Literacy? Competencies and Design Considerations. In: *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, CHI ’20, pp 1–16, <https://doi.org/10.1145/3313831.3376727>, URL <https://doi.org/10.1145/3313831.3376727>
- Lopes MM, Borlido-Santos J, Rui Vilar-Correia M, et al (2022) Hybrid Lab Network an exploratory project on STEAM for Higher Education. In: *Proceedings of the 10th International Conference on Digital and Interactive Arts*. Association for Computing Machinery, New York, NY, USA, ARTECH ’21, pp 1–6, <https://doi.org/10.1145/3483529.3483724>, URL <https://dl.acm.org/doi/10.1145/3483529.3483724>
- Lye SY, Koh JHL (2014) Review on teaching and learning of computational thinking through programming: What is next for K-12? *Computers in Human Behavior* 41:51–61. <https://doi.org/10.1016/j.chb.2014.09.012>, URL <https://www.sciencedirect.com/science/article/pii/S0747563214004634>
- Manorat P, Tuarob S, Pongpaichet S (2025) Artificial intelligence in computer programming education: A systematic literature review. *Computers and Education: Artificial Intelligence* 8:100403. <https://doi.org/10.1016/j.caeai.2025.100403>, URL <https://www.sciencedirect.com/science/article/pii/S2666920X25000438>

- Martin L (2015) The Promise of the Maker Movement for Education. *Journal of Pre-College Engineering Education Research (J-PEER)* 5(1). <https://doi.org/10.7771/2157-9288.1099>, URL <https://docs.lib.purdue.edu/jpeer/vol5/iss1/4>
- McKenzie JE, Brennan SE, Ryan RE, et al (2019) Defining the criteria for including studies and how they will be grouped for the synthesis. In: *Cochrane Handbook for Systematic Reviews of Interventions*. John Wiley & Sons, Ltd, p 33–65, <https://doi.org/10.1002/9781119536604.ch3>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119536604.ch3>, section: 3 _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781119536604.ch3>
- Murcia K, Cross E, Lowe G (2025) Young children’s computational thinking: educator pedagogy fostering children’s play and learning with a tangible coding device. *The Australian Educational Researcher* 52(2):1261–1279. <https://doi.org/10.1007/s13384-024-00762-9>, URL <https://doi.org/10.1007/s13384-024-00762-9>
- Odersky M, Spoon L, Venners B (2019) *Programming in Scala*, fourth edition edn. Artima, Walnut Creek, California
- OECD (2024) *PISA 2022 Results (Volume III): Creative Minds, Creative Schools*. PISA, OECD, <https://doi.org/10.1787/765ee8c2-en>, URL https://www.oecd.org/en/publications/pisa-2022-results-volume-iii_765ee8c2-en.html
- Olsson J, Granberg C (2024) Teacher-student interaction supporting students’ creative mathematical reasoning during problem solving using Scratch. *Mathematical Thinking and Learning* 26(3):278–305. <https://doi.org/10.1080/10986065.2022.2105567>, URL <https://doi.org/10.1080/10986065.2022.2105567>, publisher: Routledge _eprint: <https://doi.org/10.1080/10986065.2022.2105567>
- Page MJ, McKenzie JE, Bossuyt PM, et al (2021) The prisma 2020 statement: an updated guideline for reporting systematic reviews. *BMJ* 372. <https://doi.org/10.1136/bmj.n71>, URL <https://www.bmj.com/content/372/bmj.n71>, <https://www.bmj.com/content/372/bmj.n71.full.pdf>
- Paszke A, Gross S, Massa F, et al (2019) PyTorch: An Imperative Style, High-Performance Deep Learning Library. <https://doi.org/10.48550/arXiv.1912.01703>, URL <http://arxiv.org/abs/1912.01703>, arXiv:1912.01703 [cs]
- Pietriková E, Ádám N, Baláz A (2025) Game-Creative Learning in Programming Courses Over 15 Years. *Acta Polytechnica Hungarica* 22(1):123–141. <https://doi.org/10.12700/APH.22.1.2025.1.7>, URL https://acta.uni-obuda.hu/Pietrikova_Adam_Balaz_153.pdf
- Plass JL, Homer BD, Kinzer CK (2015) Foundations of Game-Based Learning. *Educational Psychologist* 50(4):258–283. <https://doi.org/10.1080/00461520.2015.1122533>, URL <https://www.tandfonline.com/doi/abs/10.1080/00461520.2015.1122533>, publisher: Routledge _eprint:

<https://www.tandfonline.com/doi/pdf/10.1080/00461520.2015.1122533>

- Quigley CF, Herro D, Jamil FM (2017) Developing a Conceptual Model of STEAM Teaching Practices. *School Science and Mathematics* 117(1-2):1–12. <https://doi.org/10.1111/ssm.12201>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1111/ssm.12201>, _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/ssm.12201>
- Raschka S, Patterson J, Nolet C (2020) Machine Learning in Python: Main Developments and Technology Trends in Data Science, Machine Learning, and Artificial Intelligence. *Information* 11(4):193. <https://doi.org/10.3390/info11040193>, URL <https://www.mdpi.com/2078-2489/11/4/193>, publisher: Multidisciplinary Digital Publishing Institute
- Resnick M, Maloney J, Monroy-Hernández A, et al (2009) Scratch: programming for all. *Commun ACM* 52(11):60–67. <https://doi.org/10.1145/1592761.1592779>, URL <https://dl.acm.org/doi/10.1145/1592761.1592779>
- Rouse R, Rouse AG (2022) Taking the maker movement to school: A systematic review of preK-12 school-based makerspace research. *Educational Research Review* 35:100413. <https://doi.org/10.1016/j.edurev.2021.100413>, URL <https://www.sciencedirect.com/science/article/pii/S1747938X21000361>
- Rousouliotis M, Vasileiou M, Manos N, et al (2024) Employing an underwater vehicle in education as a learning tool for Python programming. *Computer Applications in Engineering Education* 32(1):e22693. <https://doi.org/10.1002/cae.22693>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/cae.22693>, _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/cae.22693>
- Runco MA, Jaeger GJ (2012) The Standard Definition of Creativity. *Creativity Research Journal* 24(1):92–96. <https://doi.org/10.1080/10400419.2012.650092>, URL <https://doi.org/10.1080/10400419.2012.650092>, publisher: Routledge _eprint: <https://doi.org/10.1080/10400419.2012.650092>
- Sari U, Çelik H, Pektaş HM, et al (2022) Effects of STEM-focused Arduino practical activities on problem-solving and entrepreneurship skills. *Australasian Journal of Educational Technology* 38(3):140–154. <https://doi.org/10.14742/ajet.7293>, URL <https://ajet.org.au/index.php/AJET/article/view/7293>
- Schad M, Jones WM (2020) The Maker Movement and Education: A Systematic Review of the Literature. *Journal of Research on Technology in Education* 52(1):65–78. <https://doi.org/10.1080/15391523.2019.1688739>, URL <https://doi.org/10.1080/15391523.2019.1688739>, publisher: Routledge _eprint: <https://doi.org/10.1080/15391523.2019.1688739>
- Shute VJ, Sun C, Asbell-Clarke J (2017) Demystifying computational thinking. *Educational Research Review* 22:142–158. <https://doi.org/10.1016/j.edurev.2017.09.003>, URL <https://www.sciencedirect.com/science/article/pii/S1747938X17300350>

- Smit R, Schmid R, Robin N (2025) Experiencing enjoyment in visual programming tasks promotes self-efficacy and reduces the gender gap. *British Journal of Educational Technology* 56(3):1231–1247. <https://doi.org/10.1111/bjet.13523>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1111/bjet.13523>, _eprint: <https://bera-journals.onlinelibrary.wiley.com/doi/pdf/10.1111/bjet.13523>
- Stein G, Jean D, Kittani S, et al (2025) A Novice-Friendly and Accessible Networked Educational Robotics Simulation Platform. *Education Sciences* 15(2):198. <https://doi.org/10.3390/educsci15020198>, URL <https://www.mdpi.com/2227-7102/15/2/198>, publisher: Multidisciplinary Digital Publishing Institute
- Su J, Yang W (2022) Artificial intelligence in early childhood education: A scoping review. *Computers and Education: Artificial Intelligence* 3:100049. <https://doi.org/10.1016/j.caeai.2022.100049>, URL <https://www.sciencedirect.com/science/article/pii/S2666920X22000042>
- Su SW, Chen LX, Yuan SM, et al (2024) Cultivating Creativity and Improving Coding Skills in Primary School Students via Domain-General and Domain-Specific Learning Scaffoldings. *Education Sciences* 14(7):695. <https://doi.org/10.3390/educsci14070695>, URL <https://www.mdpi.com/2227-7102/14/7/695>, publisher: Multidisciplinary Digital Publishing Institute
- Terroso T, Pinto M (2022) Programming for Non-Programmers: An Approach Using Creative Coding in Higher Education. In: Simões A, Silva JC (eds) *Third International Computer Programming Education Conference (ICPEC 2022)*, Open Access Series in Informatics (OASIs), vol 102. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, Dagstuhl, Germany, pp 13:1–13:8, <https://doi.org/10.4230/OASIs.ICPEC.2022.13>, URL <https://drops.dagstuhl.de/entities/document/10.4230/OASIs.ICPEC.2022.13>, iSSN: 2190-6807
- Thomas J, Kneale D, McKenzie JE, et al (2019) Determining the scope of the review and the questions it will address. In: *Cochrane Handbook for Systematic Reviews of Interventions*. John Wiley & Sons, Ltd, p 13–31, <https://doi.org/10.1002/9781119536604.ch2>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119536604.ch2>, section: 2 _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781119536604.ch2>
- Thongkoo K, Daungcharone K, Thanyaphongphat J, et al (2024) Ubiquitous Learning Management Using Collaborative Inquiry-based Approach for Programming Course: A Case Study of 3 Universities. In: *Proceedings of the 2023 7th International Conference on Education and E-Learning*. Association for Computing Machinery, New York, NY, USA, ICEEL '23, pp 116–123, <https://doi.org/10.1145/3637989.3638002>, URL <https://dl.acm.org/doi/10.1145/3637989.3638002>
- Türkmen G, Sezen A, Şengül G (2024) Comparative Analysis of Programming Languages Utilized in Artificial Intelligence Applications: Features, Performance, and Suitability. *International Journal of Computational and Experimental Science*

- and Engineering 10(3). <https://doi.org/10.22399/ijcesen.342>, URL <https://www.ijcesen.com/index.php/ijcesen/article/view/342>
- Ulger K (2018) The Effect of Problem-Based Learning on the Creative Thinking and Critical Thinking Disposition of Students in Visual Arts Education. *Interdisciplinary Journal of Problem-Based Learning* 12(1). <https://doi.org/10.7771/1541-5015.1649>, URL <https://docs.lib.purdue.edu/ijpbl/vol12/iss1/10>
- Vadaparty A, Zingaro D, Smith IV DH, et al (2024) CS1-LLM: Integrating LLMs into CS1 Instruction. In: *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*. Association for Computing Machinery, New York, NY, USA, ITiCSE 2024, pp 297–303, <https://doi.org/10.1145/3649217.3653584>, URL <https://dl.acm.org/doi/10.1145/3649217.3653584>
- Wang HY, Huang I, Hwang GJ (2016) Comparison of the effects of project-based computer programming activities between mathematics-gifted students and average students. *Journal of Computers in Education* 3(1):33–45. <https://doi.org/10.1007/s40692-015-0047-9>, URL <https://doi.org/10.1007/s40692-015-0047-9>
- Weintrop D, Wilensky U (2015) To block or not to block, that is the question: students' perceptions of blocks-based programming. In: *Proceedings of the 14th International Conference on Interaction Design and Children*. Association for Computing Machinery, New York, NY, USA, IDC '15, pp 199–208, <https://doi.org/10.1145/2771839.2771860>, URL <https://doi.org/10.1145/2771839.2771860>
- Weintrop D, Beheshti E, Horn M, et al (2016) Defining Computational Thinking for Mathematics and Science Classrooms. *Journal of Science Education and Technology* 25(1):127–147. <https://doi.org/10.1007/s10956-015-9581-5>, URL <https://doi.org/10.1007/s10956-015-9581-5>
- Wing JM (2006) Computational thinking. *Commun ACM* 49(3):33–35. <https://doi.org/10.1145/1118178.1118215>, URL <https://dl.acm.org/doi/10.1145/1118178.1118215>
- Yakman G (2008) STEAM Education: an overview of creating a model of integrative education. Thesis, Virginia Polytechnic and State University
- Ye H, Ng OL, Leung A (2024) Examining mathematics teachers' creative actions in programming-based mathematical activities. *ZDM - Mathematics Education* 56(4):695–711. <https://doi.org/10.1007/s11858-024-01579-7>, URL <https://doi.org/10.1007/s11858-024-01579-7>
- Yunianto W, Jarvis D, Lavicza Z, et al (2025) Primary school students' problem-solving strategies in creating artworks with GeoGebra: Integrating computational thinking skills into mathematics and visual arts lessons. *LUMAT: International Journal on Math, Science and Technology Education* 13(2):1–1. <https://doi.org/10.31129/LUMAT.13.2.2547>, URL <https://journals.helsinki.fi/lumat/article/view/>

Supplementary Files

This is a list of supplementary files associated with this preprint. Click to download.

- [include.xlsx](#)